IIT Madras

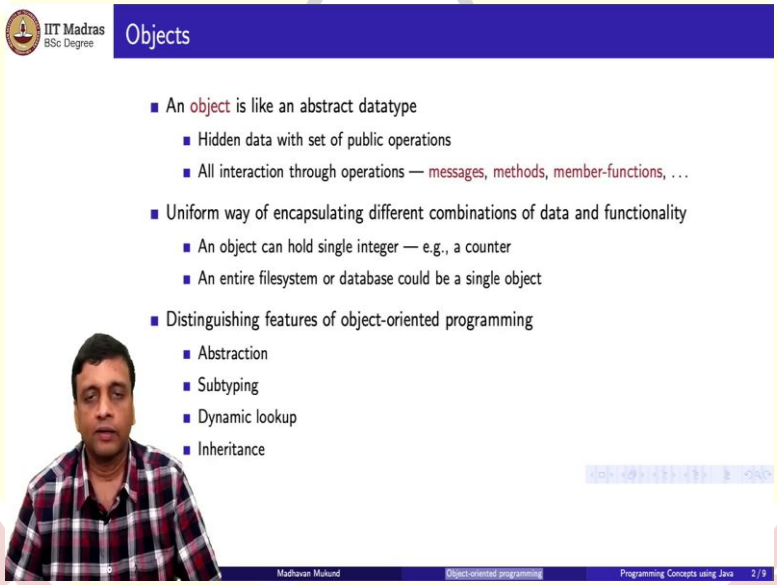
ONLINE DEGREE

Programming Concepts Using Java
Online Degree Programme
B. Sc in Programming and Data Science
Diploma Level
Prof. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute

Lecture 5
Object Oriented Programming

So, having seen what an abstract data type is let us try to see how object-oriented programming which is the main focus of this course relates to the notion of an abstract data type.

(Refer Slide Time: 00:25)



So, an object is essentially the same thing as an abstract data type in the sense that it has some hidden internal data along with a set of public operations. And as we would expect from an abstract data type all the interaction that we do with such an object is through the operations. And in object-oriented programming there are many different names given to these operations.

So, very often they are called messages or methods or in some programming languages they are called member functions and so on. But they all mean the same thing these are essentially the public functions which we use to manipulate the data inside an object or to find out information about the data inside an object. So, an object provides a uniform way of encapsulating different combinations of data and functionality.

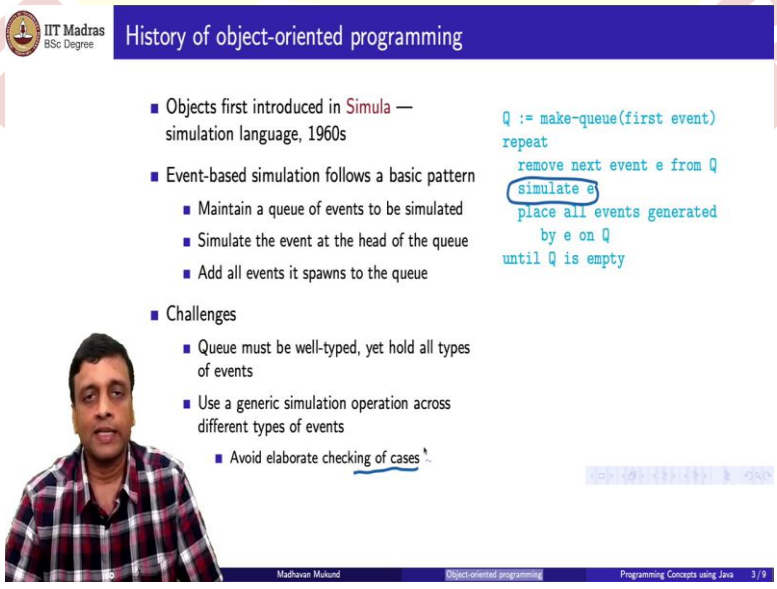
So, it does not mean that an object has to be very complicated for instance supposing we want to represent a counter. A counter is essentially just an integer with an increment operation. So, an object can consist of just an integer inside and a public function to increment this integer or to reset it to zero or at the other extreme we could have a large volume of data inside an object.

For instance, we could have the entire contents of our disk an entire file system or we could have a large database with multiple tables and we could manipulate this entire database or this entire file system as a single object where the data is the information contained in the file system of the database and the operations are those that we choose to allow on this object.

So, we will look at four distinguishing features that characterize object oriented programming. The first of course is abstraction which is related to this notion of an abstract data type but where object oriented programming goes beyond just mere abstract data types is the is in the fact that it allows what is called sub typing. So, there is a hierarchy of types.

And along with sub typing comes an important idea called dynamic lookup which we shall explain and there is also a related idea to subtyping called inheritance which is to do with reusing implementations.

(Refer Slide Time: 02:40)



So, before we get into these four specific aspects of object oriented programming let us see where object oriented programming first arose. So, the first programming language which actually introduced objects in the way that we use it now is a language called simula which came out in the 1960s and the name suggests simula was used as a simulation language.

So, what would you do typically in a simulation you have a system which you are abstractly running. So, the system has certain activities or events and you keep running these events and exploring what happens as a result of these events and exploring the evolution of the system. So, the typical run of a simulation starts like this you start off with an initial event which you put into a queue. So, this is a queue of events that remain to be simulated.

So, at every point you take the head of the queue the first event e in the queue and then you simulate it right. So, you do whatever effect that event has on the environment that you are trying to simulate and in turn this event might trigger other events. So, you trigger these events by putting them in the queue. So, when their turn comes you will explore their effect.

So, though in a real life system these events will be happening concurrently simultaneously whatever in a simulated system you execute them in a particular order following a queue. So, what is the problem with this well the first thing is that the queue is a regular data structure. So, normally when we have a queue or a stack or an array or any sequence that is well typed all the elements of this data structure may play the same type. Now events in such a system will typically be of many different types.

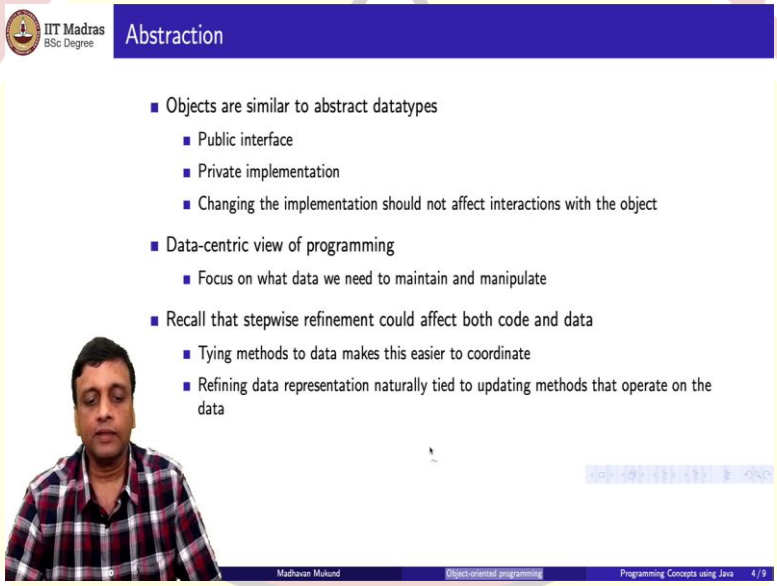
So, how are we going to accommodate all the different types of events into a well typed queue the second thing involves this step. So, when we say simulate e this means we have to perform the actions that e performs on its environment the environment that we are simulating and this will differ from event to event. So, one way to do this is to have an elaborate checking of cases.

If e is of type e1 do this if ease of type e2 do that and so on but this would be very tedious to program and very tedious to keep track of in our code. Suppose we add a new

type of event for instance we will have to change our code to add one more case. So, what we would like is that each event if it is an abstract data type of itself it has with it a method or a function or a member function or whatever which tells it how to simulate itself.

So, we just activate this function on the event and each event in some sense knows how to simulate itself. So, this is the kind of model that simula was trying to create and this is the reason why simula needed objects. It needed some way of telling each event to simulate itself without going into this elaborate description of which type of event it is at the time of calling the function.

(Refer Slide Time: 05:37)



So, with that background let us get back to these four aspects of object oriented programming that we introduced in the beginning. So, the first is abstraction. So, we said that objects are similar to abstract data types. So, they will have a public interface and a private implementation. So, usually the private implementation is the data that is stored inside we do not normally allow an external piece of code to access this data directly.

And there will be public functions which access this data manipulated updated report quantities based on it and so on. And the whole principle of separating out the public interface from the private implementation is that if we for whatever reason change the private implementation we might make it more efficient more scalable or any such thing from a functionality point of view the user of the public interface should not see any different.

So, any code that uses this object should not have to change just because we have changed the way this object is implemented. So, this is like a component in a system that we talked about when we were doing this kind of refinement. So, when we elaborate a component and make it more complex the remaining components continue to interact with it in very much the same way.

So, the reason that object oriented programming is slightly different from the kind of programming that we are familiar with is that normally when we write programs traditionally we concentrate on the control flow. We concentrate on what is to be done we concentrate on the operations we define functions and then the data is used to keep track of the state of the computation.

Now object oriented programming reverses in some sense the focus and says let us first understand what data we are trying to keep track of and based on this we ask what we need to do to manipulate this data. So, what are the requirements for the data? So, this makes a lot of sense if you are building a system for example like a banking system. So, you need to ask yourself if I am building a banking system what is the kind of data I need to keep track of.

So, I will need to keep track of information about bank accounts about customers maybe about transactions and then we start asking how to keep this information what format we should keep it and what are the allowed interactions and operations on these data things. So, this is the data centric view of programming which gives rise is object oriented methodology.

So, one thing to keep in mind is that we generally have to build a complex program by breaking it down into smaller tasks. So, we call this stepwise refinement. So, we saw that you could step wise refine a task by breaking it down to sub tasks. So, you make a larger more abstract task into more concrete tasks. So, instead of saying print a table you would exclusively write a loop saying for each element in the table print it and so on.

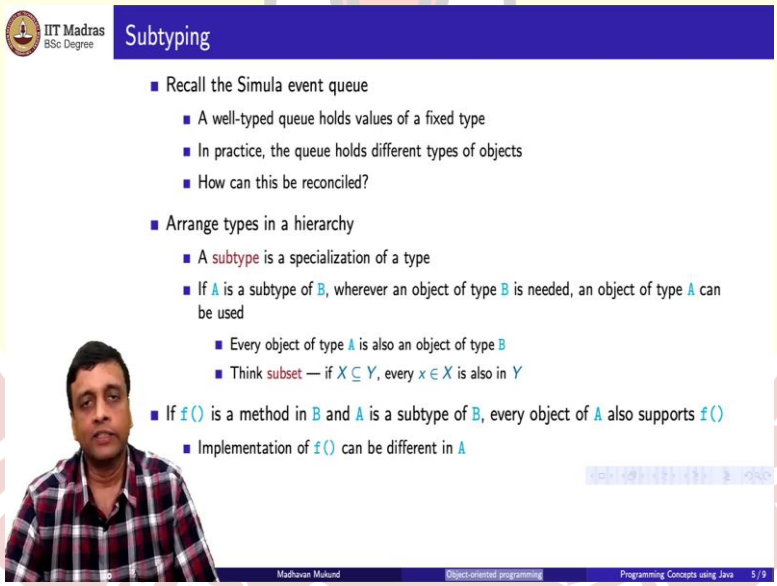
And at the same time we may also need to change the data structure that we are using. So, we saw this in the bank account example where we had to move from keeping just

the balance to keeping the transactions. And there the difficulty was that once we move to keeping transactions along with the balance all the functions which manipulated bank accounts needed to change their behaviour.

So, we needed to somehow keep track of this change across multiple functions. Now if we are doing this in this abstract data type or data centric world all the functions that manipulate a piece of data will be actually coded along with that data. So, when you change the internals of an object if you need to change the functionality of some of the functions it comes automatically because you are all you know that the internals are changing because they are all in one place.

So, from a kind of you might call it a bureaucratic administrative point of view it is much easier to track the changes that occur because of changes in your data refinement on the functions that you are writing.

(Refer Slide Time: 09:23)



So, let us move to this aspect of object oriented program which takes us slightly away from just abstract data types. So, let us get back to the simula event queue. So, we said that a similar event queue will hold values of a fixed type but in practice what is in the queue is a collection of this different types of objects. So, we have a kind of heterogeneous collection of objects.

So, how do we reconcile this combination of wanting to have a cue which has a fixed well defined type at the same time allowing it to hold a collection of different types of

objects. So, the solution to this is to allow what is called a hierarchy of types. So, we can say that one type is a sub type of another type. So, what do we mean by sub type well we mean that it is a more specialized version of that type.

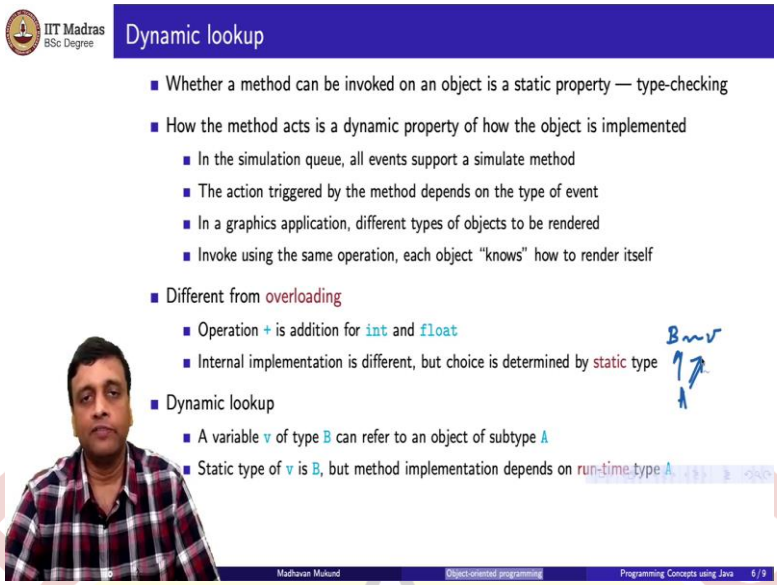
So, whenever a is a subtype of B in any context where we require a B we can also use A. So, a sub type has as many capabilities as B and more. So, if you are confused about the terminology sub type you should think about subsets. If x is a subset of y it means that anything that is an x is also a y but not everything that is a y is an x. So, everything that is an x is a y. So, similarly everything that is an A is also a B but there are some B's that are not A's.

So, this is what sub type means. So, A is more specialized than B but it can do everything that B is expected to do. So, in particular if there is a function f that you can call on a type B and A is a sub type of that type B then that function f will also be valid on A. So, syntactically if you are just thinking in terms of type it is legal to call function f on A if you have an A instead of a B.

So, if you know that a is a sub type of B you are guaranteed that the functions that are applied applicable to B will also be applicable to A. But now is a big difference the implementation of f could be different in A and this is what is happening in the simula queue. So, you have a kind of higher level maybe event which has a simulate function and every concrete event which sits inside the queue is a sub type of this event.

So, all these event types also have a simulate function but each of them has a simulate function which is specific to itself. So, in this way when I call an event from the queue and I ask it to simulate itself it will call its own implementation of simulate. So, the call will be legal because of the type and the action will depend on what type of event it is.

(Refer Slide Time: 12:06)



So, this aspect is what is called dynamic lookup. So, whether a method or a function can be invoked on an object is a static property. So, this is like type checking. So, once we have defined type B we know all the functions that are associated with B and if A is a sub type of B and this is something which is syntactic it is part of the program code you will declare A to be a subtype of B then all the functions of B will naturally be available for A.

However how these functions behave when you invoke them on objects of type A will be dynamically determined by the type of object it is. So, I might have more than one thing which is a subtype I have more than one type of event. So, each subtype will possibly have its own implementation of these functions. So, in our simulation queue example every event supports a simulate method but the action that the simulate method triggers on in terms of updating the environment depends on the type of event.

In a different example supposing we have a program which is a graphical program. So, we are trying to manipulate some objects on the screen there will be some shapes and there will be some other things which we want to put together. So, imagine a kind of image editing program. So, here we can change these objects and then periodically we might want them to redraw themselves according to the changes.

So, each object can just be told to redraw itself. So, redraw is like simulate and each object depending on the nature of the object knows how to redraw itself. So, this is again best done in this object oriented thing where we have this dynamic lookup. So, dynamic

lookup says you are you have the right to invoke a function on an object because the object satisfies the correct typing requirement and what that actual operation performs depends on what that object is at runtime.

So, this is different from what we sometimes see in programming languages which is called overloaded where we use the same operator or the same function on different types. So, a simple example is when we have this plus operator. So, plus operator in almost any programming language can be applied to two ends or to two floats now from a mathematical perspective an int or a float is a number and we know that plus represents addition.

So, mathematically really there is no difference between plus on ins and plus on floats but of course we know that representation wise because of the way that bits are stored and all that inserts stored differently from floats. So, the algorithmic operation of adding two integers stored in binary is different from the algorithmic operation required to add two floats stored in binary. Nevertheless when we use it in our program we do not want to write two separate things a plus I and a plus f.

So, we just use a plus and which plus is used and which internal algorithm is used to add these values depends on the static type associated with the variables we are trying to add. So, if it is integers it will use the integer algorithm for which addition in terms of bit strings if it is float it will use the float algorithm but this is static. So, overloading happens when we apply different functions with the same name to different types and the type of the value at as it is declared determines which interpretation of this function is going to be used.

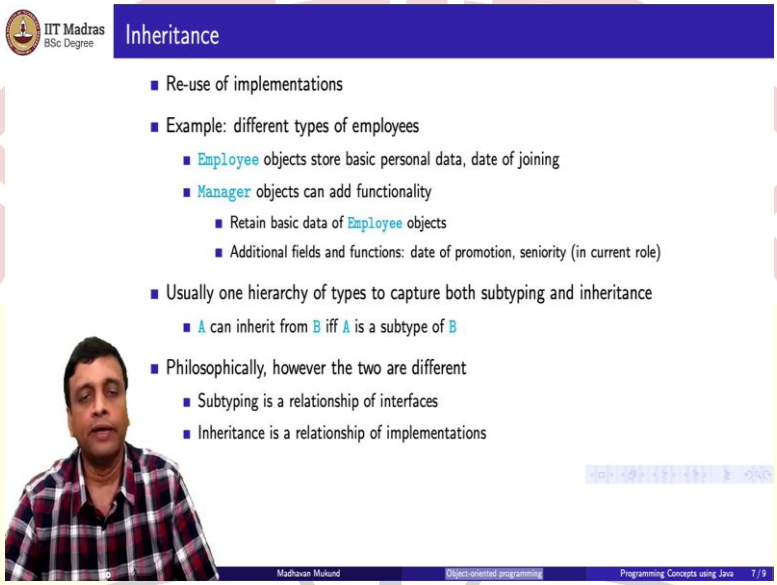
Now in dynamic lookup this is done at runtime. So, one of the properties of sub typing as you will see is that you can declare a variable to be of the parent type and assign it to be an object of the lower type. So, for instance if A is a subtype of B then so, A is you should think of it as A is a subtype of B and now I have declared a v to B of this type I can actually assign v to point to an object of type A.

So, although statically the compiler believes that v is of type B at runtime the value stored in B is actually of type A. So, now if I take a function which is defined for B and I

execute it on the object stored in V it will not take the definition based on the type of v which is capital B rather it will take it based on the type of the object that is stored. So, at this time there is an A object stored in B.

So, A has its own definition of whatever function I want and that will be dynamically chosen. So, this is what dynamic lookup is all about.

(Refer Slide Time: 16:25)



Finally let us look at what is called inheritance. So, inheritance is a fundamental property of software development which is that if you have something you have already written you would not like to rewrite it. So, all over the place we try to reuse the code that we have used. So, in particular when we are defining data types and data structures and so on we would like to reuse implementations to the extent possible.

So, here is an example supposing that we are trying to keep track of different types of employees in our organization. So, we might start with a basic type employee an abstract data type which stores information about a basic type employee which has basic personal data like their name and various things may be also their date of joining. Now we might want to keep extra information about some types of employees for instance we might have managers whom we want to treat differently in some parts of the code.

So, for a manager we would like to add some functionality. So, maybe we might want to have some information about the date of promotion maybe all employees who are not managers have the same designation but managers come in different types there are

assistant managers and there are senior managers and so on. So, we might want to know their date of promotion their exact role how long they have been in the current role and so on but every manager is still an employee.

So, every manager actually still has personal data had a rate of joining and so on. So, whatever we have defined for an employee can carry over to a manager we do not have to rewrite those parts. The way of calculating for example the date of current seniority in terms of years of service will be to subtract the date of joining from today's date. Now this is the same whether the person is an employee of a normal type or a manager but if I want seniority in the current role it refers to date of promotion which does not make sense for employees it makes sense for managers.

So, in this way manager has additional functionality but for the functionality that overlaps with an employee we would not like to rewrite it. So, we would like to reuse that code. So, normally when we create hierarchies of types in a programming language we only have one way of defining this hierarchy. So, we already saw that we have this hierarchy referring to sub typing A is a sub type of B which says that in every context where we want a B we can substitute an A instead.

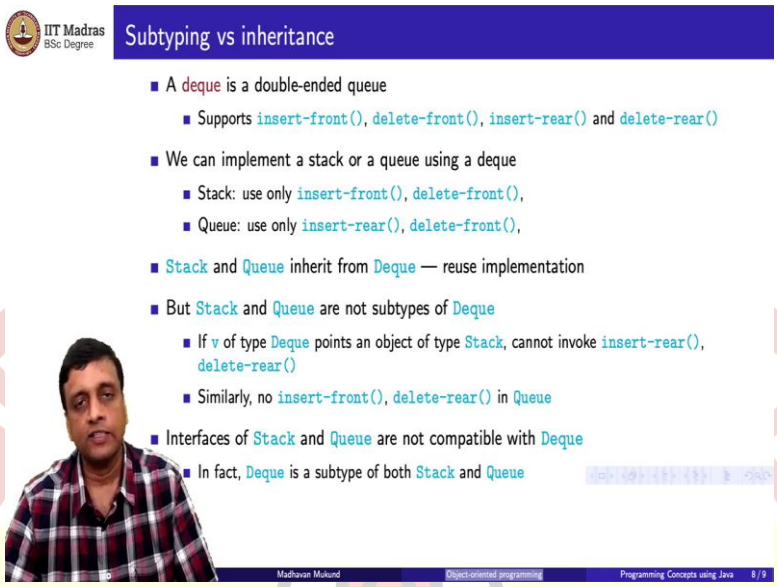
So, A is a more specialized version of B. So, this goes hand in hand in most programming languages with inheritance. So, if we want to copy some code from B or not copy but actually implicitly reuse it then A should be sitting below B in this hierarchy. So, A can inherit from B. So, in the earlier case you would have employee as a parent type right and we would have manager as A sub type.

So, manager would inherit from employee however philosophically these are two different things. So, sub typing tells us something about whether a function is legal or not it is a static property. If there is an f which is defined on the parent type B then A must be able to do f right how a does f is a dynamic property but the fact that a can do an f is a property of its static type.

The fact that it is sitting under B however. So, it is really a property of interfaces you know which functions are legal and which functions are not whereas inheritance is a relationship of implementations I am reusing something which I already have in B in the

code for A. So, if B has defined some variables then A can make use of those variables because A will also have access to those variables.

(Refer Slide Time: 19:58)



So, to explore this difference between sub typing and inheritance. So, here is a nice example. So, a double ended queue or a deque is a queue in which you can add and remove from either end. So, let us say that this is the front and this is the rear right. So, I can insert in the front or I can remove the element in the front I can delete in the front or I can insert in the rear and I can delete from the rear.

So, it has still some limited functionality its not a list I cannot arbitrarily put something in the middle of a deque but I can do it at either end I can either insert at this end or that end I can remove it. So, it has four different operations two inserts and two deletes. Now using this I can easily implement a stack or a queue. So, remember that a stack in this terminology would actually always operate only at the front at the top right.

So, this is a stack and a queue on the other hand would insert at the rear and delete from the front you join at the end of the queue and you leave at the beginning of the queue now if we had a deque then these two data structures that we need to implement can be implemented by just choosing two out of the four operations that a deque supports and for the rest the deque has a sequence of values and a stack and a queue have a sequence of values. So, we can implement a stack or a queue easily using a deque.

So, in the sense that we are talking about reusing implementations we can say that stack and queue are classes which inherit or objects which inherit from the object deque. But if you think about interfaces they are not sub types. So, supposing I want to declare that stack is a sub type of deque. So, remember that if A is a sub type of B and if f is A function aware. So, if A is a sub type of B right and if I have a function here then I must also allow that function here this is a static property of subtypes.

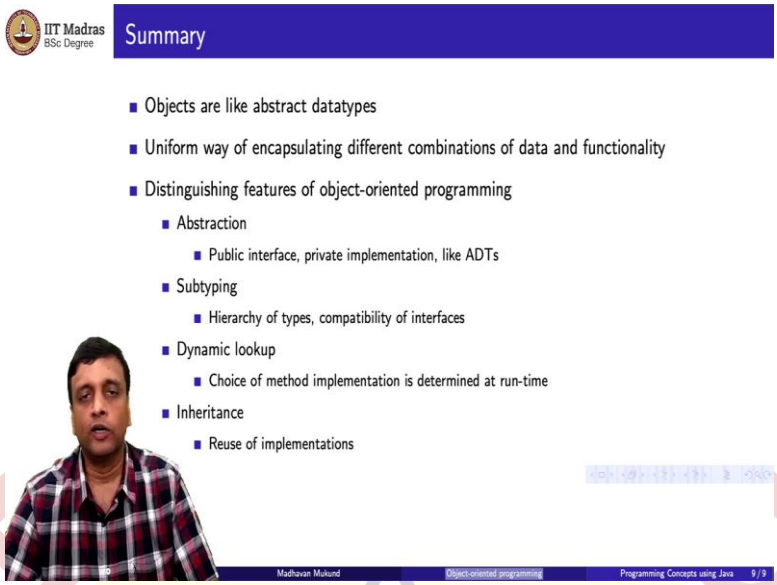
So now we look at this deque right. So, deque is sitting on top and it has four functions. Now supposing I claim that stack is a sub type but here I only have two functions right I only have the insert front and the insert delete front. So, if I if I ask a stack to delete rear this is a legal thing for a deque but if stack were a sub type of deque it would also have to support this operation right but it does not.

So, a stack has less functionality than a deque. So, it cannot be a sub type of deque even though it inherits from deque in the sense that we can use an implementation of deque to implement sag and the same is true of queue right q also only two of these things it has an insert rear and a delete front. So, if I want the opposite ones if I want to insert in the front and delete in the rear I cannot do it on a queue.

So, in this way we can see that the interfaces of stack and queue are not compatible with deque. So, in fact is the other way around right. So, deque technically can be treated as a sub type because everything that a stack can do a deque can do. The two functions on stack are both available in deque the two functions in queue are both available in deque. So, if at all there is a subtyping relation it is the other way around.

So, this is the real difference between sub typing and inheritance. So, one is a question of whether you are reusing an implementation the other is whether or not the interfaces are preserved.

(Refer Slide Time: 23:15)



So, to summarize objects are very similar to abstract data types and objects provide us a uniform way of encapsulating different combinations of data and functionality. So, what we said is the data that is stored on object can be very simple it can be a single integer like a counter it can be very complicated it can be a database. But what really distinguishes object oriented programming from abstract data types is this notion of sub typing.

So, obviously objects have abstraction like an abstract data type. So, you would like to separate out a public interface from a private implementation but you also allow yourself to arrange these types in a hierarchy. And by allowing yourself to arrange these types in a hierarchy you can talk about a variable of a higher type containing a value of a sub type and this allows you to create this heterogeneous queue for example that is used in simula.

And when you do that it goes hand in hand with this idea of dynamic lookup. So, you have a collection of objects which all share a common parent type. So, they all have a common capability but the exact nature of this capability depends on the specific object. So, this is what is called dynamic lookup when I invoke a function f even if the variable is pointing to the parent type B

the actual subtype A will determine what the effect of f is on that particular object and finally we saw that very often sub typing coincides with an idea of inheritance. So, when A is a subtype of B we can also reuse parts of the implementation of B in a but the

example with the deque and the queue and the stack shows that technically speaking reuse of implementations is not always compatible with a subtyping.

But in practice what we will find is that in the languages that we looked at python and the language we are going to look at java. There is only one way to organize these types there is only one hierarchy we do not have a subtyping hierarchy separately and an implementation hierarchy separately. So, we have to live with whatever we can do within that one thing. So, usually we will go with the sub typing hierarchy and we will do whatever reuse of implementation is allowed there.